

WIOF Foundation Hardening & Platform Readiness Plan - v4

Purpose: Give Kiro / Copilot an implementation-grade program for strengthening WIOF before major product expansion.

Status: This is the complete foundation *planning baseline*. After foundation hardening, WIOF enters a dedicated Brand & Experience Alignment phase before major new product features are built. The foundation is not considered complete in production until all P0/P1 gates are implemented, tested, deployed to staging, validated, and then promoted to production.

Baseline reviewed: WIOF Angular/Ionic source, Firebase configuration, Firestore rules/indexes, Storage rules, Cloud Functions, deployment files, current reset-engagement implementation, current TODO/improvement material, previous WIOF roadmap documents, and the ClimatEnlighten brand guide.

External verification: Firebase security guidance and current official documentation reviewed 2026-08-26. See the Research Basis section at the end.

1. Product North Star

WIOF is the primary platform. Its long-term loop is:

Discover -> Understand -> Act -> Track -> Improve -> Measure

The platform should help an individual improve everyday environmental/lifestyle behavior while making progress and impact understandable.

The foundation must therefore support, safely and efficiently:

- identity and authorization
- privacy and user-controlled deletion/reset
- reliable activity/event capture
- trusted derived metrics
- scalable data access
- strong performance and failure tolerance
- admin/product analytics
- coherent information architecture
- simple onboarding and guided discovery
- future recommendations
- future measurable action/impact tracking
- future Planet Passport achievements
- future anonymous collective-impact aggregation

Explicit strategic constraints

1. **WIOF remains the prime product.**

2. **ClimatEnlighten is a separate website/platform.** WIOF may recommend or deep-link to its courses; do not tightly couple the two systems in the foundation phase.
3. **No social/community network in this phase.** No public profiles, feeds, messaging, following, social graph, or user-to-user interaction.
4. **Collective impact is still a future requirement.** Store and aggregate individual activity safely so WIOF can later show anonymous totals without exposing individuals.
5. **Do not replace Firebase.** Strengthen the existing Angular/Ionic + Firebase architecture.
6. **Do not implement gamification now.** Make the data model future-ready but only build the Passport/achievement layer after the foundation is trustworthy.
7. **No major redesign that destabilizes current journeys.** Security fixes and architecture changes must be backward compatible or accompanied by controlled migration.
8. **Staging first.** Firebase rules, functions, schema migrations and data migrations must be validated against the staging project before production.
9. **Guest-first access is a hard product requirement.** Public WIOF exploration must remain available without authentication. Authentication adds memory, personalization, ownership and continuity; it must not become a gate for the core public website unless a specific security, privacy or abuse-prevention requirement is documented.
10. **Do not create persistent personal journey data for guests.** Guest interactions may contribute to appropriately configured aggregate analytics, but persistent user-owned activity, journey history, Passport state and personal impact belong to authenticated identities.
11. **Brand alignment follows foundation hardening.** After security/data/platform readiness gates pass, perform a controlled WIOF visual and UX alignment pass using the approved ClimatEnlighten/WIOF brand system. Do not mix this visual migration into security remediation work.
12. **Brand alignment must preserve product usability.** Apply the new palette, typography, logo rules, visual language, content hierarchy and imagery principles consistently, but do not force decorative elements where they do not improve clarity.

2.1 Post-Foundation Brand & Experience Alignment Gate

Brand alignment is intentionally a separate phase after foundation hardening. The goal is to make the existing WIOF website visually and experientially consistent with the new WIOF/ClimatEnlighten brand system without destabilizing the product.

Scope

- inventory current use of logos, colors, typography, cards, buttons, icons, illustrations and imagery
- establish shared design tokens for approved colors and typography
- align WIOF logo usage with the approved WIOF rules and ensure the new ClimatEnlighten visual family is used consistently where applicable
- replace legacy/near-match colors with approved values
- use Aleo for headings/titles and Noto Sans for readable body copy where the brand system calls for them
- establish consistent CTA hierarchy and spacing
- align illustration/graphic treatment with the approved flat, graphic visual language
- review page hierarchy, navigation, cards and content density for clarity
- ensure responsive/mobile parity
- audit accessibility, contrast, focus states and reduced-motion behavior during the visual migration
- update reusable components first, then page-level exceptions
- visually regression-test representative public, authenticated and admin flows

Important brand principle

The visual system should make something clearer. Do not add rays, suns, circles, waves, decorative motifs or other brand elements merely to fill space. The brand guide explicitly says to start with the idea and use its visual vocabulary only where it strengthens that idea. filecite turn0file0 L333-L350

Acceptance criteria

- WIOF uses an approved, documented design token system instead of scattered hard-coded colors.
- Logo variants follow the approved background/contrast rules.
- Typography is consistent across reusable UI components.
- Primary/secondary CTA hierarchy is consistent.
- Public pages, authenticated pages and admin pages have a coherent visual system while preserving their different functional needs.
- No major user journey is functionally broken by the visual migration.
- Mobile and desktop layouts are visually and functionally verified.
- Accessibility checks are part of the acceptance gate.
- Brand migration is reversible through normal version control and can be rolled out incrementally.

2. Definition of Done for the Foundation

The foundation is considered ready only when all of the following are true:

- A browser client cannot escalate itself to admin.
- Admin authorization comes from a trusted authorization source.
- System-derived engagement counters cannot be client-forged.
- Raw private user data is not publicly enumerable.
- Poll raw responses are private.
- Subscription records cannot be enumerated by public clients.
- User engagement reset is reliable, idempotent, and truthful about its result.
- Every user-owned collection is classified as resettable/retained/account-delete-only/aggregated.
- Activity events follow a documented schema and semantic rules.
- Unique events cannot duplicate through concurrent requests.
- Raw events remain available for audit/analytics while derived metrics are generated separately.
- Dashboard queries do not require downloading unbounded raw history.
- Firestore and Storage rules have emulator regression tests in CI.
- App Check is introduced through staged monitoring and controlled enforcement.
- Sensitive configuration/secrets are not exposed in source or client bundles.
- The site has a clearly documented information architecture and cross-linking model.
- First-time users can understand what WIOF offers and what to do next without mandatory friction.
- Major content types have an owner, taxonomy, related-content strategy and next-step behavior.
- Observability exists for security, functions, write failures and key product events.
- Foundation changes do not break existing public content, Google Sign-In, favorites, EQ history or admin workflows.
- Guest users can continue to access the full intended public WIOF experience without being forced to sign in.
- Signing in does not remove or reduce any public capability; it adds persistence, personalization and continuity.
- Guest-only activity is not silently converted into a permanent personal journey without an explicit identity transition and defined data policy.

3. P0 - Security Remediation

3.1 Fix self-service admin-role escalation

Current risk

The supplied Firestore rules allow a user to create their own `users/{uid}` document using an ownership check while the authorization function uses a `role`

field in that document. The client currently creates the profile as `public`, but client behavior is not a security boundary.

Target

A normal browser client must never be able to assign itself an administrative role.

Preferred architecture:

- Firebase Authentication establishes identity.
- Trusted backend logic creates/maintains privileged authorization state.
- Admin authorization uses a trusted Firebase Auth custom claim such as `admin: true` or another tightly controlled role claim.
- Firestore and Storage rules consume the trusted auth claim for admin access.
- `users.role` may remain temporarily for UI/compatibility but must not remain the security authority.

Migration

1. Inventory all admin users.
2. Set trusted admin claims for existing admins using a server-side process.
3. Update Firestore and Storage rules.
4. Update application-side role checks.
5. Preserve `users.role` only as non-authoritative compatibility data during transition.
6. Add tests that attempt forged `role: admin` create/update operations.
7. Remove any remaining client write path for privileged role fields.

Acceptance criteria

- A standard user cannot create an admin profile.
- A standard user cannot change their role.
- A standard user cannot access admin collections.
- A standard user cannot publish/delete protected media.
- Existing admins retain access after migration.
- Rules tests cover both create and update escalation attempts.

3.2 Protect system-derived user fields

Current profile fields include engagement metrics such as:

- `loginCount`
- `lastLogin`
- `daysVisited`
- `currentStreak`

- `savedBlogsCount`

These should be treated as server/system state, not normal profile preferences.

User-editable

- `displayName`
- `photoURL`
- `preferredElements`

System-managed

- login counts
- visit counts
- streaks
- completion counts
- activity aggregates
- future action completion counts
- future Passport achievements
- future points
- future impact calculations

A browser client must not directly set these fields.

3.3 Fix public access to raw poll responses

The supplied implementation has poll documents containing response-level data such as email/IP/selected option while the client-facing rule permits public reads.

Required target

Separate public poll configuration/results from private raw responses.

Suggested conceptual split:

- `polls` or `PollQuestions`: public question/configuration and sanitized aggregate results
- `poll_responses`: private raw responses

Raw response data should be accessible only to trusted/admin paths when necessary.

Do not expose raw email/IP data to public clients.

Acceptance criteria

- Anonymous users cannot enumerate raw votes.

- Authenticated users cannot read other users' raw votes.
 - Raw email/IP values are not publicly readable.
 - Admin reporting continues to work.
 - Public poll results are aggregate/sanitized only.
-

3.4 Tighten subscription lookup privacy

The current public email-existence lookup pattern must not become an indirect subscriber-enumeration API.

Preferred approach:

- use a deterministic subscriber document key where suitable, or
- perform duplicate checking server-side.

Public clients must not be granted general collection list access to subscriber data.

3.5 Audit every Firestore path and Storage path

Create a permission matrix covering every production collection and storage prefix.

For each resource document:

- public read?
- authenticated read?
- owner read/write?
- admin read/write?
- server-only write?
- sensitive fields?
- resettable?
- account-delete behavior?

Do not accept “the frontend never calls it” as a security control.

4. P0 - Authentication, Session and App Protection

4.1 Preserve Google Sign-In and idle timeout

Existing Google authentication and idle timeout should continue to work.

Regression test:

- sign-in
- profile creation
- sign-out
- route guards
- idle timeout
- multi-account switching
- stale session handling

4.2 Guest access and authenticated value boundary

The current and future product must preserve a clear separation between public exploration and private personalization.

Guest users

Guests should be able to use essentially all public WIOF capabilities, including:

- browse the website
- read blogs/articles
- watch public videos
- explore Earth, Energy, Air, Water and Spirit
- use public environmental tools/widgets
- view calendar/events
- explore public polls/content
- browse courses/course information
- use other public capabilities that do not require a personal identity

Authenticated users

Authentication adds capabilities such as:

- saved/favorite content
- personal activity history
- My Journey
- EQ history
- streaks and progress
- future Action completion history
- future Planet Passport state
- future personal impact
- future personalized recommendations

Required architectural rule

Do not solve product personalization by turning public content into authenticated-only content. The public site remains the top-of-funnel experience. Login is a value proposition around continuity and memory.

Guest-to-user transition

The application may use temporary, privacy-appropriate session context to improve a guest's current experience. If a guest signs in, any decision to migrate pre-authentication context into persistent user-owned data must be explicit, documented and tested. Do not silently create a historical personal journey for anonymous visitors.

4.3 App Check

Introduce Firebase App Check for the web after staging tests.

Rollout:

1. staging configuration
2. local/debug workflow
3. monitoring mode
4. investigate invalid traffic
5. staging enforcement
6. production enforcement

Do not enable production enforcement blindly.

4.4 Secret/configuration audit

Review all environment/config values and bundled frontend assets.

Classify every value:

- public configuration
- sensitive configuration
- server-only secret

Anything that is genuinely secret must not ship in the Angular bundle.

5. P0 - Privacy and User-Controlled Data Reset

5.1 Keep the existing Clear My Engagement Data feature

The current Profile capability that lets an authenticated user remove engagement data must be preserved. It is an existing product capability, not a missing feature. The hardening work is to make its deletion contract reliable, complete, idempotent and accurately communicated.

The current Profile/Settings flow already provides a useful privacy feature that clears engagement data while preserving the account.

This is **not a missing feature**. It should be hardened.

Current behavior includes:

- delete activity records
- delete saved-content records
- reset engagement counters
- keep account/profile identity

5.2 Separate reset from account deletion

Reset engagement data

Keeps account identity and intended profile data, while removing resettable engagement state.

Delete account

Future, separate capability for full user-account/data deletion according to the platform's retention/privacy policy.

Do not merge the two.

5.3 Move destructive reset orchestration to trusted backend logic

The current client performs activity deletion, saved-content deletion and profile reset sequentially. A failure part way through can leave partial deletion, while the UI currently says that nothing was changed after a failed step.

Implement a trusted Cloud Function or equivalent authenticated backend endpoint.

Requirements:

- verify caller identity
- verify caller owns the UID
- enumerate resettable records
- delete in controlled batches
- reset system-derived metrics
- include future derived metrics/Passport state when classified as resettable
- return structured status
- be safe to retry
- do not retain deleted engagement content in function logs

5.4 Make reset idempotent

Running reset twice should converge to the same clean state.

5.5 Inventory resettable data

Create a `RESETTABLE_USER_DATA` contract covering every user-owned collection.

Each future collection must declare exactly one of:

- `resettable`
- `retained on engagement reset`
- `deleted only on account deletion`
- `anonymized/aggregated`
- `system/admin content - not user-owned`

This contract must be updated whenever a new user-data collection is introduced.

5.6 Fix user-facing reset messages

The UI must distinguish:

- `completed`
- `already clean / no-op`
- `retry required after failure`
- `operational error`

Never claim “nothing changed” unless the backend can prove that no destructive step succeeded.

6. P1 - Canonical Event / Activity Architecture

6.1 Keep `activity_log` as raw event history

The existing activity collection is the correct starting point for long-term behavioral analytics.

Do not replace it with a giant user profile document.

6.2 Establish an event schema

Conceptual fields:

- `eventId`
- `userId`
- `eventType`
- `eventVersion`
- `occurredAt`
- `calendarDay`
- `contentType`
- `contentId`

- source
- schemaVersion
- typed event metadata

Keep the schema controlled. Do not let arbitrary clients store arbitrary fields forever.

6.3 Event categories

Engagement

- session_started
- content_opened
- content_completed
- bookmark_added
- bookmark_removed

Learning

- course_opened
- learning_item_completed
- quiz_completed
- eq_completed

Action

Future-ready:

- action_started
- action_completed
- action_repeated
- evidence_submitted

Recommendation

Future-ready:

- recommendation_shown
- recommendation_opened
- recommendation_completed

System/privacy

- data_reset_requested
- data_reset_completed
- account_deleted

Do not implement every event now; standardize naming/versioning now.

7. P1 - Event Integrity, Deduplication and Semantics

The current read-before-write activity deduplication can race under concurrent requests.

Use deterministic identity where uniqueness is required

Examples:

- daily visit: `userId + eventType + calendarDay`
- lifetime blog completion: `userId + eventType + contentId`
- repeatable action: append a new event with a unique event ID

Do not apply one deduplication rule to every event type.

Define event semantics explicitly:

- once per day
- once per content
- repeatable
- immutable
- mutable until completion

For security-sensitive or reward-bearing events, prefer trusted backend creation/validation.

8. P1 - Raw Events vs Derived Metrics

Raw events should remain the source of truth.

Add a derived user metrics layer such as:

`user_metrics/{uid}`

Potential values:

- `blogsOpened`
- `qualityReads`
- `videosCompleted`
- `pollsCompleted`
- `eqCompletions`
- `actionsCompleted`
- `currentStreak`
- `lifetimeActions`
- `elementDistribution`
- `lastActiveAt`
- `estimatedImpact` summaries

Derived metrics must not be treated as primary evidence. They are materialized views for fast UX/admin analytics.

If a derived value is wrong, it must be possible to recompute it from raw events.

9. P1 - Saved Content Data Integrity

Current saved-content logic uses query-before-delete and query-based ownership.

Improve it with a deterministic identity strategy where practical:

`userId + contentId`

This reduces duplicate risk and simplifies delete/unsave operations.

Keep pagination for the UI.

Do not keep `savedBlogsCount` as a client-authoritative counter. Either derive it or update it through trusted aggregation.

10. P1 - Firestore and Storage Rules Testing

Create automated Emulator Suite tests using `@firebase/rules-unit-testing` or equivalent current Firebase testing tooling.

Users

- anonymous access denied where required
- user owns own profile
- cross-user profile modification denied
- role change denied
- forged admin create denied
- admin access allowed

Saved content

- owner allowed
- other user denied
- unauthenticated denied

Activity

- own event create allowed
- own event read allowed where intended

- cross-user read denied
- post-create mutation denied where events are immutable
- malformed event denied
- spoofed `userId` denied

Poll responses

- public raw read denied
- allowed vote creation only
- cross-user raw reads denied
- admin/trusted read allowed

Subscriptions

- public subscribe allowed only through the intended path
- collection enumeration denied
- admin management allowed

Storage

- owner avatar upload allowed within size/type rules
- invalid content type denied
- oversize denied
- other-user overwrite denied
- non-admin protected-media write denied

Run the rules tests in CI.

11. P1 - Analytics and Observability Foundation

WIOF needs two complementary analytics systems.

Product analytics

Track:

- acquisition/source where legally and technically appropriate
- sign-up conversion
- returning users
- retention
- content engagement
- completion rates
- EQ usage
- action adoption later
- recommendation performance later

- course click-through to ClimatEnlighten later

Operational analytics

Track:

- function failures
- failed writes
- authentication errors
- rules denials where safely observable
- third-party API failures
- slow operations
- Storage failures
- App Check failures

Do not place personal/sensitive content into logs.

12. P1 - Performance and Scalability

12.1 Do not scan raw history on every dashboard load

Use derived metrics/materialized summaries for common dashboard values.

12.2 Query discipline

Every new query must have:

- bounded result size
- correct index
- pagination where the result can grow
- clear ownership/authorization
- cost awareness

12.3 Avoid N+1 reads

Do not retrieve one Firestore document per card when a batched/aggregated representation is possible.

12.4 Cache carefully

Cache public content where useful, but never allow stale authorization or private-user data to leak across accounts.

12.5 Failure tolerance

Third-party services such as AQI/weather/food APIs should fail gracefully without making the whole dashboard unavailable.

13. P1 - Information Architecture and Content Connectedness

Technical foundations are not enough. WIOF must also feel coherent.

Current content types include blogs, videos, news, conversations, In Focus, polls, calendar, courses, Take Action, tools/widgets, EQ and My Journey.

Users should not have to understand the internal CMS taxonomy to know where to go.

Target mental model

Discover -> Learn -> Act -> Track -> Improve

The existing five elements (Earth, Energy, Air, Water, Spirit) remain an important content taxonomy, but they should not compete with the primary user journey.

Required foundation work

1. Audit all current navigation.
2. Map each page/content type to one primary purpose.
3. Remove duplicate or competing navigation concepts.
4. Define a canonical taxonomy for elements, topics, content types and action categories.
5. Define a related-content model.
6. Define a page-level “next best step” pattern.
7. Define consistent breadcrumbs/section context where useful.
8. Ensure content can naturally connect:
 - event -> fact -> article -> video -> action
 - blog -> related video -> action -> course
 - EQ -> relevant activity/content
 - action -> impact -> next action
9. Prevent dead-end pages.

Content metadata to future-proof

Where applicable, content should eventually support:

- element

- `topic`
- `contentType`
- `difficulty`
- `audience`
- `relatedContentIds`
- `relatedActionIds`
- `relatedCourseIds`
- `source/verification`
- `publishedAt`
- `status`

Do not introduce all fields blindly; define a common taxonomy first.

14. P1 - Onboarding and Guided First Experience

WIOF contains many capabilities. A first-time user may not understand what to do next.

Create a lightweight onboarding foundation, not a mandatory tutorial maze.

Guest first visit

A short welcome/tour can explain:

1. Explore WIOF
2. Discover environmental topics
3. Understand yourself through EQ and tools
4. Take simple actions
5. Track your journey
6. Eventually grow a Planet Passport

Authenticated onboarding

After Google Sign-In, optionally ask:

- what topics interest you
- what type of experience you want
- whether they want to take EQ now or later

Then provide a simple first-week progression.

Principles

- skippable
- resumable

- no more than a few steps at once
- do not block a user who only wants to read one article
- clearly show where the user is and what to do next
- store onboarding state as user preference/progress, not as arbitrary activity spam

Future dashboard progression

A new user may eventually see:

- Explore your first topic
- Read your first story
- Complete your first video
- Take EQ
- Complete your first action
- Unlock your first Passport milestone

This creates a guided experience without turning WIOF into a game-first product.

15. P1 - Calendar / “Today on WIOF” Future Foundation

This is a product idea to keep on the roadmap, not an immediate build blocker.

Potential experience when a user lands on WIOF:

Today on WIOF

- today’s relevant calendar events
- one concise fact or question per selected event
- related content/action links
- optional close/skip

Potential future chain:

Calendar event -> curiosity fact -> related content -> action -> Passport/impact

Do not create a heavy modal on every visit. Use frequency rules, dismissal state, relevance and accessibility.

16. P2 - Data Governance for Future Impact and Passport

Before implementing environmental impact or achievements, create explicit metric governance.

Every future impact metric must define:

- source
- formula
- assumptions
- unit
- time basis
- confidence/estimate status
- version of methodology
- whether user-provided or system-derived
- whether it is resettable

Never turn an unverified estimate into a precise-looking fact.

Future Passport data model requirements

A future Passport should be derived from trusted events and achievement rules.

Conceptual model:

raw event -> validated metric -> achievement rule -> Passport milestone

Examples:

- trees planted
- energy-conscious actions
- recycling actions
- learning milestones
- EQ milestones
- streak milestones
- healthy-food streaks
- mobility actions

The achievement layer must not be client-authoritative.

17. P2 - Privacy-by-Design for Future Collective Impact

WIOF does not need a social network to show collective impact.

Future architecture should support:

individual raw events -> trusted aggregation -> anonymous WIOF totals

Do not expose:

- public user profiles
- individual environmental histories
- detailed behavior of another person
- personal identifiers in public impact views

Collective metrics should be aggregate-first and methodology-backed.

18. Technical Refactoring Guidance for Kiro / Copilot

AI coding agents must follow these rules while implementing the plan:

1. **Inspect before changing.** Do not invent collections, routes or services that already exist.
2. **Prefer minimal, targeted changes.** Avoid broad rewrites.
3. **Do not weaken Security Rules to make tests pass.** Fix application behavior or rules design instead.
4. **Do not trust client-side guards as authorization.** Firebase Rules and trusted backend logic are the actual security boundary.
5. **Do not move secrets into Angular environment files if they are server secrets.** Frontend configuration is public.
6. **Do not create client-authoritative points, badges, impact or admin roles.**
7. **Every data migration must be reversible or have a tested roll-back/recovery procedure.**
8. **Write/extend tests with every security/data-model change.**
9. **Use staging Firebase first.**
10. **Preserve existing routes and user workflows unless a security defect requires change.**
11. **Document schema/version changes.**
12. **Keep raw events recoverable.** Do not make an aggregate the only evidence.
13. **Use feature flags or guarded rollout for user-visible changes.**
14. **Do not implement community/social features in this foundation phase.**

19. Recommended Implementation Order

Phase A - Security lock-down

- role escalation fix
- system-field write protection
- poll privacy redesign
- subscription enumeration fix
- full Firestore/Storage permission matrix
- emulator security tests
- App Check staging rollout

Gate: security tests pass; staging audit shows no known P0 issue.

Phase B - Privacy/reset reliability

- trusted reset function
- idempotent reset
- resettable-data registry
- accurate UI states
- regression tests

Gate: repeated reset is safe; no resettable data remains afterward.

Phase C - Event/data foundation

- canonical event schema
- event versioning
- deterministic IDs/idempotency
- saved-content uniqueness
- separation of raw vs derived metrics

Gate: raw events are reproducible and metrics can be recomputed.

Phase D - Analytics/observability/performance

- derived user metrics
- bounded queries
- dashboard optimization
- operational error monitoring
- product analytics baseline

Gate: dashboard remains fast with large synthetic activity datasets.

Phase E - UX foundation

- IA audit
- taxonomy
- related-content model

- next-step patterns
- onboarding state model
- lightweight first-time journey

Gate: a new user can explain what WIOF is and what to do next without training.

20. Future Feature Dependencies

After the foundation, the following features should consume the foundation rather than bypass it:

Planet Passport

Depends on: - trusted activity events - achievement rules - derived metrics - privacy reset contract

Action engine

Depends on: - action schema - event semantics - trusted completion - methodology-backed impact

Personal recommendations

Depends on: - activity history - user preferences - content taxonomy - recommendation analytics

ClimatEnlighten recommendations

Depends on: - content/topic signals - recommendation layer - safe deep links

Collective impact

Depends on: - trusted event aggregation - privacy classification - impact methodology

21. Deliverables Kiro / Copilot Should Produce

Implementation should not be considered complete because code “builds”.

The agent should produce:

1. updated Firestore rules
2. updated Storage rules where required

3. automated rules tests
 4. trusted backend functions for privileged/reset logic
 5. event schema/types and validation
 6. migration scripts where needed
 7. derived metric/aggregation logic
 8. updated unit/integration tests
 9. CI test commands/documentation
 10. deployment/runbook updates
 11. migration and rollback notes
 12. security findings resolved/remaining register
 13. information-architecture decision record
 14. onboarding decision record
 15. verification checklist with staging results
-

22. Manual Verification Checklist Before Production

Security

- ☐ normal user cannot become admin
- ☐ user cannot modify protected counters
- ☐ raw poll responses private
- ☐ subscribers cannot be enumerated
- ☐ protected Storage paths verified
- ☐ App Check status verified

Privacy

- ☐ engagement reset complete
- ☐ repeated reset safe
- ☐ no user engagement remains after reset
- ☐ intended profile identity retained
- ☐ resettable data inventory is current

Data integrity

- ☐ duplicate event scenarios tested
- ☐ event schema validated
- ☐ aggregate metrics recomputable
- ☐ saved content duplicates prevented

Performance

- ☐ large activity dataset tested

- ☐ dashboard query bounded
- ☐ no major N+1 reads
- ☐ third-party failure behavior tested

Guest access

- public pages and public content remain reachable without authentication
- guest users are not forced through onboarding
- sign-in is an optional continuity/personalization upgrade
- no persistent personal journey is created for anonymous visitors without a defined transition policy

UX foundation

- ☐ navigation hierarchy documented
- ☐ content types have clear ownership/purpose
- ☐ related content paths work
- ☐ first-time experience is understandable
- ☐ onboarding is skippable and resumable

Deployment

- ☐ staging deployment verified
- ☐ production backup/rollback procedure ready
- ☐ rules tests run in CI
- ☐ migration scripts rehearsed
- ☐ monitoring active after release

23. Research Basis

The foundation plan was also checked against current official guidance on 2026-08-26:

- Firebase Security Rules are an independent security boundary and should be tested with the Local Emulator Suite before production deployment.
- Firebase recommends treating rules as part of the data/schema design and adding automated rules tests to CI.
- Firebase documents App Check as an additional control to help ensure requests come from the genuine app.
- Firebase documents custom Auth claims as a mechanism for trusted role-based access.
- Firestore server/admin libraries bypass Firestore Security Rules, so server-side functions must be protected by IAM/trusted execution.

Primary official sources:

- <https://firebase.google.com/docs/rules/>
 - <https://firebase.google.com/docs/firestore/security/test-rules-emulator>
 - <https://firebase.google.com/docs/firestore/security/insecure-rules>
 - <https://firebase.google.com/docs/firestore/security/overview>
 - <https://firebase.google.com/support/guides/security-checklist>
-

24. Final Principle

Make WIOF trustworthy before making it clever.

First build:

**Secure identity -> reliable events -> trusted metrics -> scalable data
-> understandable product -> measurable outcomes**

Then build:

actions -> Passport -> personalization -> impact -> collective impact

The purpose of this order is not to delay innovation. It is to make every future enhancement cheaper, safer, more measurable and easier to evolve.